

Modern Disturbance Observers: Online Compensation of Sim-to-Real Mismatch from Traditional Models to World Models

The International Journal of Robotics Research

XX(X):1–14

©The Author(s) 2026

Reprints and permission:

sagepub.co.uk/journalsPermissions.nav

DOI: 10.1177/ToBeAssigned

www.sagepub.com/



Tianqi Zhu¹, Jun Yang², Jianliang Mao³, Cong Li², and Shihua Li¹

Abstract

Robotic controllers increasingly rely on analytical models, simulators, cost-query interfaces, and learned world models. However, physical deployment can deviate from nominal assumptions, and additional disturbances may arise even when the model itself is accurate. In control systems, disturbance observers (DOB) are widely used to estimate such unmeasured effects from nominal models and measured feedback. Classical DOB formulations are generally built around explicit plant models. This paper develops modern disturbance observers (MDOB), extending the same principle to sampling-based model predictive control (MPC) with models accessed through state rollouts or cost queries. MDOB separates two observable channels: state-effect disturbances, for which the feedback state differs from its prediction, and cost disturbances, for which the same query state receives different costs as the perceived environment changes. The resulting estimates compensate future state rollouts and cost evaluation online without plant inversion or model retraining. Simulation and real-robot studies span analytical systems, moving-obstacle navigation, latent world-model control, and dual-arm obstacle avoidance, demonstrating a common disturbance-observation interface from traditional models to world models.

Keywords

Modern disturbance observers, sim-to-real, sampling-based model predictive control, world models

1 Introduction

From early model-based control to modern robotics built on simulators and learned models, deployment has always involved discrepancies between internal models and physical reality. In classical systems, these discrepancies appear as parameter uncertainty, unmodeled dynamics, and external disturbances (Ohnishi et al. 1996; Chen et al. 2016); in contemporary robotics, the same problem is often discussed as the sim-to-real gap (OpenAI et al. 2020; Jaquier et al. 2025). Such discrepancies can degrade tracking performance and closed-loop stability. If a disturbance is measurable, feedforward compensation can often reduce its influence. In practice, the relevant disturbances are often unmeasured or entangled with model uncertainty. Classical DOB formulations estimate these effects in systems described by explicit plant or low-dimensional state-space models (Ohnishi et al. 1996; Chen et al. 2016; Sariyildiz et al. 2020). Conventional MPC operates within the same model-based setting, optimizing behavior over a finite horizon using explicit dynamics and constraints (Garcia et al. 1989; Mayne et al. 2000; Ostafew et al. 2016). DOB estimates can

therefore be incorporated into its prediction and optimization loop, for example by correcting output predictions (Yang et al. 2015).

However, modern robotic systems increasingly use sampling-based MPC when the dynamics or costs are highly nonlinear, nonsmooth, or accessible only through simulation. Sampling-based MPC evaluates candidate action sequences through forward rollouts. Methods such as model predictive path integral control (MPPI) (Williams et al. 2018a; Carius et al. 2022; Koptev et al. 2024) and the

¹School of Automation, Key Laboratory of Measurement and Control of CSE, Ministry of Education, Southeast University, Nanjing 210096, China

²Robotics and Autonomous Systems Thrust, Systems Hub, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou 511453, China

³Faculty of Artificial Intelligence, Shanghai University of Electric Power, Shanghai 200090, China

Corresponding author:

Jun Yang, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou 511453, China; Jianliang Mao, Shanghai University of Electric Power, Shanghai 200090, China.

Email: jy@hkust-gz.edu.cn (Jun Yang); jl.mao@shiep.edu.cn (Jianliang Mao)

cross-entropy method (CEM) (Rubinstein 1997; Kobilarov 2012; Pinneri et al. 2021) rank these candidates using scalar trajectory costs. Because they require only forward rollouts and scalar costs, these methods can use nonlinear simulators without analytic derivatives. At the same time, rollout evaluation has expanded beyond analytical dynamics to perception-derived cost maps and signed-distance fields (Hornung et al. 2013; Oleynikova et al. 2017; Zucker et al. 2013), while world models (Hafner et al. 2019, 2020) learn action-conditioned latent dynamics for prediction and planning. This flexibility allows MPC to use a broader range of models, but it also makes disturbance estimation less direct. The resulting interfaces may provide neither an invertible plant representation nor a clearly defined input-equivalent disturbance channel. Unmodeled effects are therefore revealed through deviations in the feedback state or changes in the costs assigned to queried states, for example after contact, perception updates, or obstacle motion. The controller observes these effects only after executing an action and receiving feedback. Taken together, these changes leave a mismatch between the information required by classical DOB formulations and the interfaces exposed by modern predictive models.

To address this mismatch, this paper extends the DOB principle to simulators, neural predictors, world models, and other models exposed through rollout or cost-query interfaces. We refer to each member of this family as a **modern model**. Without requiring a plant inverse, MDOB compares predicted and feedback states or repeated costs at the same query states to estimate the discrepancy observable through the controller interface. The resulting estimates compensate the next state rollout or cost evaluation. MDOB therefore targets locally persistent, task-relevant mismatch rather than complete identification of the sim-to-real gap. The main contributions are:

- 1) A new interface-level disturbance representation is introduced for modern models. It separates observable deployment mismatch into a state-effect disturbance that changes rollout states and a cost disturbance that changes the costs returned for fixed query states, extending disturbance observation from state evolution to environment-dependent cost evaluation.
- 2) Based on this representation, MDOB estimates both disturbance channels online and incorporates them into future state propagation and cost evaluation. Compared with inverse-model DOB formulations, MDOB requires only forward rollouts or cost queries and does not require plant inversion or model retraining.

- 3) The framework is evaluated through three simulation studies and two representative real-robot demonstrations spanning analytical dynamics, moving-obstacle navigation, latent world-model control, and dual-arm obstacle avoidance. Source code, configurations, and experiment scripts are released to support reproducibility.

The remainder of this article is organized as follows. Section 2 reviews related work. Section 3 introduces the classical DOB formulation used as a reference. Section 4 presents MDOB, including its deployment-oriented interpretation, two disturbance channels, and compensation interfaces. Section 5 reports the simulation and real-robot experiments. Section 6 concludes the article and discusses its limitations and future directions.

2 Related Work

MDOB connects four areas: disturbance observation, sampling-based MPC, sim-to-real adaptation, and world models. Disturbance observation provides the observer principle, sampling-based MPC provides rollout and cost evaluation interfaces, and sim-to-real adaptation motivates online correction under deployment mismatch. World models represent a frontier class within the modern models defined above and provide learned, action-conditioned representations for predicting future states.

2.1 DOB-Based Control

Classical DOB-based control estimates input disturbances from a nominal plant, commonly through an inverse model and low pass filtering, to improve motion control robustness (Ohnishi et al. 1996; Shim and Jo 2009; Sariyildiz et al. 2020; Park and Back 2024). Nonlinear DOB formulations and state space estimators extend this principle to nonlinear and robotic systems (Chen et al. 2000a,b; Mohammadi et al. 2013; Chen et al. 2016). Active disturbance rejection control (ADRC) and extended state observer (ESO) methods likewise estimate lumped uncertainty in feedback loops (Han 2009; Gao 2003), while DOB-enhanced predictive control incorporates disturbance estimates into prediction and optimization (Yang et al. 2015). Many classical DOB formulations assume an explicit plant model and a predefined disturbance channel. These assumptions are difficult to retain when control operates on images, latent or object centric states, or rollout distributions available only through forward simulation and cost evaluation.

2.2 Sampling-Based MPC

Sampling-based MPC optimizes candidate action sequences through forward rollouts and ranks, weights, or refits them using trajectory costs (Theodorou et al. 2010; Williams et al. 2015, 2018a; Pinneri et al. 2021). Requiring only simulated futures and scalar costs makes it suitable for nonlinear, nonsmooth, and black box models. Tube-MPPI uses MPPI to generate a nominal trajectory and an ancillary tracking controller to keep the actual state within a tube around that trajectory (Williams et al. 2018b). Robust model predictive path integral control (RMPPI) extends this architecture with augmented rollouts of the nominal and actual dynamics, tracking-aware importance sampling, and safety logic for nominal-state propagation (Gandhi et al. 2021). Parameter uncertainty can be incorporated directly into path integral control (Abraham et al. 2020), while Shield-MPPI integrates control barrier functions for safety (Yin et al. 2023).

2.3 Sim-to-Real Adaptation

The methods above address uncertainty mainly within the online control loop. Sim-to-real adaptation takes a complementary approach by changing the training distribution, system model, or simulator. Domain randomization broadens the training variation (Tobin et al. 2017), system identification calibrates the dynamics (Tan et al. 2018), and feedback driven simulator adaptation updates the simulator from real experience (Chebotar et al. 2019). Online methods can align latent representations under unknown target appearances (Yoneda et al. 2022), while ManiDreams propagates perceptual, parametric, and structural uncertainty through task specific predictive models (Wang et al. 2026). Such methods adapt the simulator, representation, policy, or predictive state, but remain tied to variations represented during training or adaptation. Greater simulator fidelity also does not guarantee that method rankings or task outcomes transfer to hardware (Kadian et al. 2020), particularly when learned states do not map directly to physical variables.

2.4 World Models for Prediction and Planning

World models learn action conditioned representations for imagined futures, from early latent models (Ha and Schmidhuber 2018) to PlaNet (Hafner et al. 2019), Dreamer (Hafner et al. 2020, 2025), and TD-MPC2 (Hansen et al. 2024). Learned dynamics for manipulation (Ai et al. 2025), broader predictive architectures (LeCun 2022), V-JEPA (Bardes et al. 2024), and PointWorld (Huang et al. 2026) extend this direction toward feature level and 3D action conditioned representations. These models

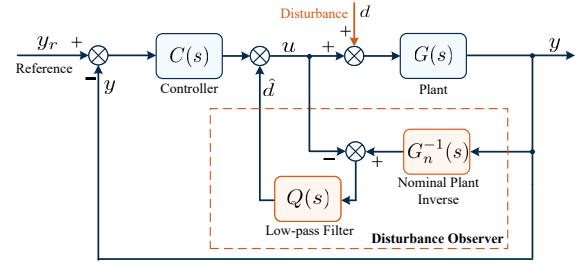


Figure 1. Classical DOB based control structure.

improve prediction, but nominal replanning does not usually preserve recurring execution mismatch as an explicit disturbance. Actuator bias, contact mismatch, object drift, and environmental change can therefore reappear as rollout error at every control cycle.

Together, these areas provide observer principles, rollout interfaces, adaptation strategies, and learned predictive representations. Existing methods do not provide a common disturbance-observation interface that treats completed state transitions and repeated cost queries as two complementary feedback channels. The next section summarizes the classical DOB structure used to develop MDOB.

3 Preliminaries

3.1 Disturbance Observer

Figure 1 shows the classical continuous-time DOB structure. For clarity, consider a single-input single-output plant with a matched input-side disturbance:

$$y(s) = G(s)(u(s) + d(s)) \quad (1)$$

where $G(s)$ is the actual physical plant, $u(s)$ is the applied control input, and $d(s)$ is an unknown disturbance entering through the same channel as the input. A standard DOB introduces a nominal model $G_n(s)$, which is the model used by the controller rather than the actual plant. The nominal inverse $G_n^{-1}(s)$ maps the measured output back to an input-equivalent quantity, which is then compared with the applied input and filtered before compensation (Ohnishi et al. 1996; Chen et al. 2016; Sariyildiz et al. 2020).

The DOB estimate is

$$\hat{d}(s) = Q(s)(G_n^{-1}(s)y(s) - u(s)), \quad (2)$$

where $Q(s)$ is a stable low-pass filter. The estimate is then used in the input channel as

$$u(s) = u_c(s) - \hat{d}(s), \quad (3)$$

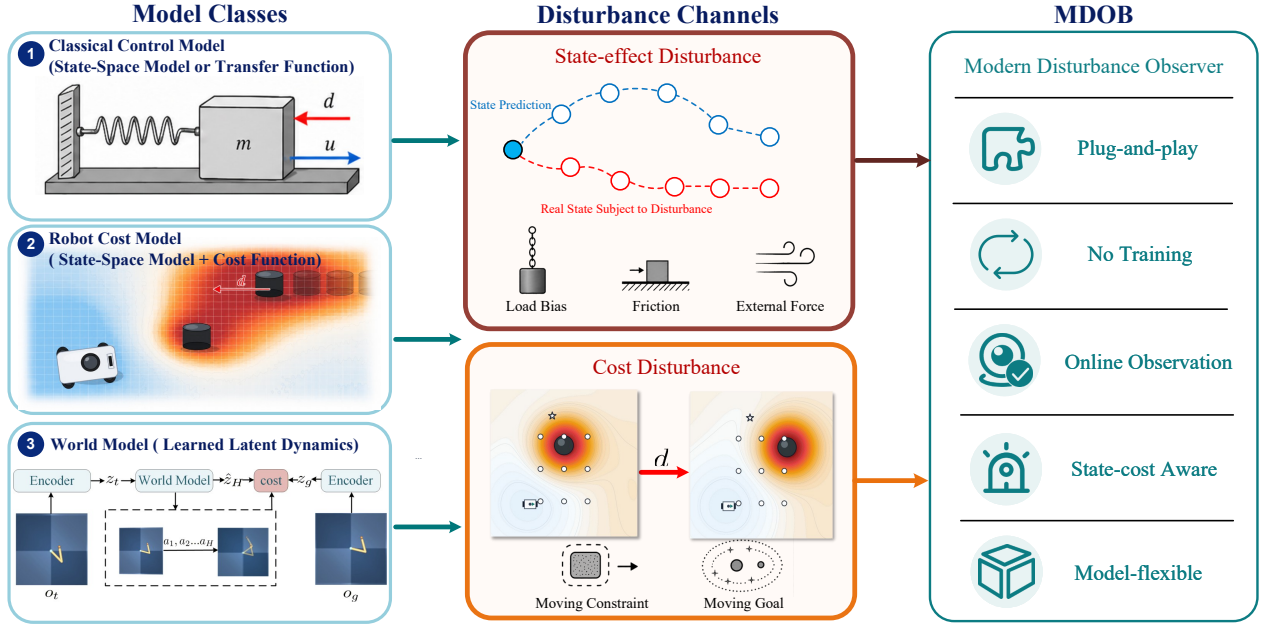


Figure 2. Overview of the MDOB framework from model classes, disturbance channels, and MDOB advantages. State-effect and cost disturbances connect classical control, robot cost, and world-model examples to a unified observer interface.

where $u_c(s)$ is the baseline controller output before disturbance compensation.

Example: discrete-time integrator. A minimal discrete-time example is a scalar integrator with a matched disturbance,

$$y_{k+1} = y_k + u_k + d_k, \quad (4)$$

with nominal prediction

$$\hat{y}_{k+1|k} = y_k + u_k. \quad (5)$$

After executing u_k and observing y_{k+1} , the one-step prediction error is

$$e_{k+1} = y_{k+1} - \hat{y}_{k+1|k} = y_{k+1} - y_k - u_k. \quad (6)$$

If the nominal model is exact and measurement noise is ignored, then $e_{k+1} = d_k$. In practice, this residual is passed through a discrete low-pass filter, for example

$$\hat{d}_k = (1 - \alpha)\hat{d}_{k-1} + \alpha e_{k+1}, \quad 0 < \alpha \leq 1, \quad (7)$$

and the next input can be compensated using the previous disturbance estimate,

$$u_k = u_{c,k} - \hat{d}_{k-1}. \quad (8)$$

This example fixes the timing used below: the residual is computed after y_{k+1} is observed, and the available estimate compensates the following control input.

4 Modern Disturbance Observers

As shown in Figure 2, MDOB extends the classical DOB principle to models accessed through forward rollouts or cost queries. It uses the mismatch between a model query and the following feedback to estimate a correction for the next control step. The estimate is defined by the interface supplied by the task; it need not identify the unique physical cause of the mismatch. MDOB therefore addresses online correction of observable, locally persistent model mismatch rather than general system identification or complete recovery of the sim-to-real gap.

4.1 Model and Forward Equations

Let $\mathcal{X}$, $\mathcal{U}$, $\mathcal{Q}$, $\mathcal{E}$, and $\mathcal{D}$ denote the controlled-state, input, cost-query, perceived-environment, and disturbance spaces. At the beginning of step k , the controller has the latest feedback state $\mathbf{x}_k^{\text{fb}}$, the perceived environment $\mathcal{E}_k$, and the cost-query function c_k . Once the feedback state is passed to the controller, we write $\mathbf{x}_k \triangleq \mathbf{x}_k^{\text{fb}}$. For a query state $\mathbf{q} \in \mathcal{Q}$,

$$c_k(\mathbf{q}) \triangleq c(\mathbf{q}, \mathcal{E}_k). \quad (9)$$

Thus $c_k(\mathbf{q})$ denotes the cost assigned to the query state $\mathbf{q}$ under the perceived environment $\mathcal{E}_k$. A robot state may contain many variables, such as joint positions and velocities, whereas $\mathbf{q}$ contains only the variables required for cost evaluation. A task-specific map extracts the query state as

$$\mathbf{q} = \Pi_q(\mathbf{x}), \quad \Pi_q : \mathcal{X} \rightarrow \mathcal{Q}. \quad (10)$$

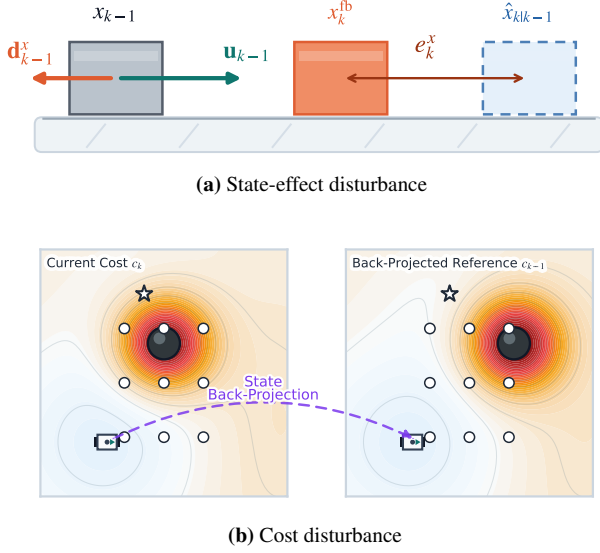


Figure 3. Two disturbance channels used by MDOB. (a) Illustration under the state-effect disturbance d_{k-1}^x . (b) Illustration under the cost disturbance d_{k-1}^c .

Examples include a vehicle pose, end-effector pose, or robot configuration passed to a collision checker.

The nominal transition model is $F_x : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$. The task also supplies a disturbance-enabled rollout interface

$$F_x^d : \mathcal{X} \times \mathcal{U} \times \mathcal{D}_x \rightarrow \mathcal{X}, \quad F_x^d(x, u, \mathbf{0}) = F_x(x, u). \quad (11)$$

This interface fixes what the state channel can estimate. For example, it may add a bias to the input or a residual to the predicted state. The resulting estimate is therefore a control-equivalent state effect, not necessarily a measured force or a unique physical parameter.

At the beginning of step k , the most recently completed transition is from $k-1$ to k . Its nominal state prediction and nominal cost-query function are

$$\hat{x}_{k|k-1} = F_x(x_{k-1}, u_{k-1}), \quad (12)$$

$$\hat{c}_{k|k-1}(q) = c_{k-1}(q) = c(q, \mathcal{E}_{k-1}). \quad (13)$$

After u_{k-1} is executed, the feedback process returns the current state x_k^{fb} and perceived environment $\mathcal{E}_k$. With an unknown state-effect disturbance, the completed transition and the current cost query can be written as

$$\begin{aligned} x_k^{fb} &= F_x^d(x_{k-1}, u_{k-1}, d_{k-1}^x), \\ c_k(q) &= c(q, \mathcal{E}_k). \end{aligned} \quad (14)$$

Thus $\hat{x}_{k|k-1}$ and $\hat{c}_{k|k-1}$ are formed before execution, whereas x_k^{fb} , $\mathcal{E}_k$, and $c_k(q)$ become available afterward. Here d_{k-1}^x denotes the unknown state-side effect acting over the completed interval.

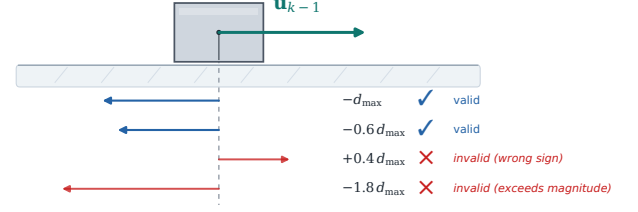


Figure 4. Physical admissibility of sampled state-effect disturbances.

4.2 Two Disturbance Channels

MDOB separates mismatch by where it appears. A state-effect disturbance changes the feedback state, for example through load, friction, contact, or actuator bias. A cost disturbance changes the cost assigned to the same query state as the perceived environment changes. This second channel is useful when a point cloud, occupancy layer, or collision model is available for queries but is not maintained as a compact dynamic state. As illustrated in Figure 3,

$$d_{k-1} = (d_{k-1}^x, d_{k-1}^c(\cdot)) \quad (15)$$

where d_{k-1}^x denotes a state-effect disturbance and $d_{k-1}^c(q)$ denotes the cost disturbance at query state q . In this work, $d_{k-1}^c(\cdot)$ is a query-dependent vector field that describes the apparent local transport rate of the queried cost, rather than the physical velocity of an individual object.

Remark 1. MDOB requires three conditions. First, the controller must provide a repeatable rollout interface or comparable cost queries across consecutive updates. Second, the selected disturbance coordinates must affect the available feedback or queried costs; components that do not affect these signals are unobservable. Third, the estimated effect must persist long enough to remain useful over the next prediction horizon. The state channel additionally requires the disturbance-enabled interface in Eq. (11). The cost channel assumes that local cost changes over one update can be approximated by short-horizon transport. Abrupt topology changes or several unresolved motions in the same neighborhood can violate this assumption.

4.3 State-Effect Disturbance Estimation

At the beginning of step k , the controller has received x_k^{fb} after executing u_{k-1} . Let $\hat{d}_{k-1}^x$ denote a candidate estimate of the unknown disturbance d_{k-1}^x that acted during this completed transition. The candidate estimate produces the prediction

$$\hat{x}_{k|k-1}(\hat{d}_{k-1}^x) = F_x^d(x_{k-1}, u_{k-1}, \hat{d}_{k-1}^x). \quad (16)$$

The corresponding state-effect residual is the difference between the feedback state and this prediction,

$$e_k^x(\hat{d}_{k-1}^x) = \mathbf{x}_k^{\text{fb}} - \hat{\mathbf{x}}_{k|k-1}(\hat{d}_{k-1}^x). \quad (17)$$

The residual in Eq. (17) scores how well each candidate disturbance explains the completed feedback transition. The disturbance estimate is obtained by minimizing

$$\begin{aligned} \hat{d}_{k-1}^{x,*} &\in \arg \min_{\hat{d}_{k-1}^x \in \mathcal{D}_x} \mathcal{L}_{k-1}^x(\hat{d}_{k-1}^x), \\ \mathcal{L}_{k-1}^x(\hat{d}_{k-1}^x) &= \underbrace{\ell_{\text{pred},k-1}^x(\hat{d}_{k-1}^x)}_{\text{current prediction error}} + \underbrace{\ell_{\text{reg},k-1}^x(\hat{d}_{k-1}^x)}_{\text{disturbance regularization}} \\ &\quad + \underbrace{\ell_{\text{adm},k-1}^x(\hat{d}_{k-1}^x)}_{\text{physical admissibility}} \\ \ell_{\text{pred},k-1}^x(\hat{d}_{k-1}^x) &= \left\| W_x e_k^x(\hat{d}_{k-1}^x) \right\|^2, \\ \ell_{\text{reg},k-1}^x(\hat{d}_{k-1}^x) &= \left\| W_d^x \hat{d}_{k-1}^x \right\|_2^2, \\ \ell_{\text{adm},k-1}^x(\hat{d}_{k-1}^x) &= R_{\text{adm}}^x(\hat{d}_{k-1}^x, \mathbf{x}_{k-1}, \mathbf{u}_{k-1}, \mathbf{x}_k^{\text{fb}}). \end{aligned} \quad (18)$$

The prediction term measures the feedback mismatch, with W_x setting the relative importance of the feedback coordinates. The regularization term discourages unnecessarily large disturbance estimates, with W_d^x weighting the disturbance coordinates. The function $R_{\text{adm}}^x \geq 0$ is a task-specific admissibility penalty for the candidate explanation of the completed transition $(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}, \mathbf{x}_k^{\text{fb}})$. It is zero for candidates consistent with known disturbance and transition constraints and positive otherwise, as illustrated in Figure 4.

Remark 2. Local identifiability. The selected disturbance coordinates are locally distinguishable only when they produce different changes in the feedback state. For a differentiable model, this condition can be checked using

$$J_{d,k-1} = \left. \frac{\partial F_x^d(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}, \mathbf{d})}{\partial \mathbf{d}} \right|_{\mathbf{d}=\hat{d}_{k-1}^x}. \quad (19)$$

Full column rank means that the selected disturbance coordinates can be distinguished locally. If the matrix is rank deficient, several disturbances may produce the same state change, and MDOB estimates only their combined observable effect. This condition is used for interpretation; the sampling estimator below does not require evaluating the derivative.

When Eq. (18) can be solved analytically, the disturbance is estimated directly. For sampled or black-box rollout models, MDOB instead draws m_x candidate disturbances for

the completed transition,

$$\hat{d}_{k-1,r}^x \sim \rho_{k-1}^x, \quad r = 1, \dots, m_x. \quad (20)$$

Here ρ_{k-1}^x generates the candidates and may be centered on the previous estimate. Each candidate is inserted into the loss in Eq. (18). Candidates with lower losses receive larger exponential weights:

$$\ell_{k-1,r}^x = \mathcal{L}_{k-1}^x(\hat{d}_{k-1,r}^x), \quad (21)$$

$$\omega_{k-1,r}^x = \frac{\exp(-\ell_{k-1,r}^x/\lambda_d)}{\sum_{q=1}^{m_x} \exp(-\ell_{k-1,q}^x/\lambda_d)}. \quad (22)$$

The normalized exponential weights define an empirical Gibbs distribution (Theodorou et al. 2010; Williams et al. 2018a), with $\lambda_d > 0$ controlling its concentration. The point estimate used in the experiments is its weighted mean,

$$\hat{d}_{k-1}^x = \sum_{r=1}^{m_x} \omega_{k-1,r}^x \hat{d}_{k-1,r}^x. \quad (23)$$

MDOB uses the weighted mean to summarize the sampled low-loss region. The estimate becomes available at step k , remains constant over the next prediction horizon, and is computed using only forward rollouts.

4.4 Cost Disturbance Estimation

The cost channel is used when environmental change is available through cost queries but not through an explicit dynamic state. Examples include a collision checker built from a point cloud, an occupancy map, or a learned risk model. The channel estimates the local transport of queried costs rather than the physical velocity of every object.

At the beginning of step k , the current and previous cost-query functions are both available. Let $\mathcal{Q}_k^c = \{\mathbf{q}_k^{(i)}\}_{i=1}^{N_c}$ be query states drawn near rollouts, safety boundaries, the feedback state, or a local workspace grid. Here N_c is the number of cost query states used at the current update; the schematic in Figure 5 uses $N_c = 9$.

Let $\hat{d}_{k-1,i}^c$ denote a candidate local cost disturbance at query state $\mathbf{q}_k^{(i)}$. One scalar query cannot determine multidimensional motion, so the candidate is evaluated over a neighborhood $\mathcal{N}_i$ of nearby query states. Over the completed interval Δt , it predicts the current cost at neighboring query state j as

$$\hat{c}_{k|k-1}^{c,(i,j)}(\hat{d}_{k-1,i}^c) = c_{k-1}(\mathbf{q}_k^{(j)} - \Delta t \hat{d}_{k-1,i}^c). \quad (24)$$

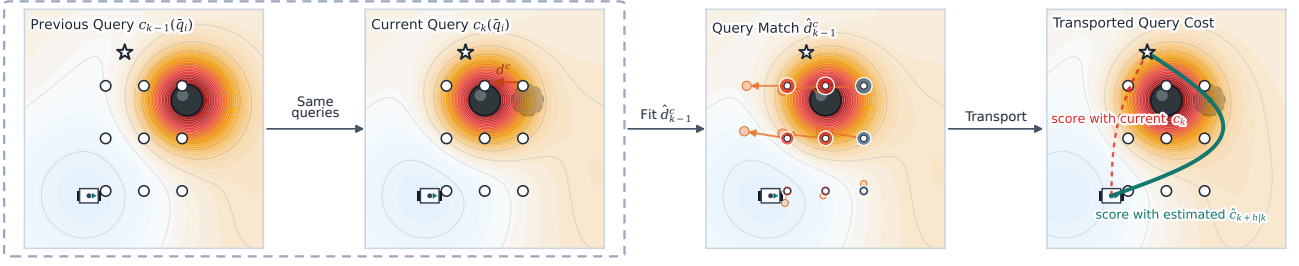


Figure 5. Causal cost disturbance estimation and planning compensation. MDOB compares fixed query states at steps $k - 1$ and k , estimates the local cost transport over the completed interval, and transports the current cost query when scoring future rollouts.

The corresponding cost residual is

$$e_{k,i,j}^c \left(\hat{d}_{k-1,i}^c \right) = c_k(\mathbf{q}_k^{(j)}) - \hat{c}_{k|k-1}^{c,(i,j)} \left(\hat{d}_{k-1,i}^c \right). \quad (25)$$

The candidate explains the local cost change when these residuals are small. The local cost disturbance estimate is obtained by minimizing

$$\begin{aligned} \hat{d}_{k-1,i}^{c,*} &\in \arg \min_{\hat{d}_{k-1,i}^c \in \mathcal{D}_c} \mathcal{L}_{k-1,i}^c(\hat{d}_{k-1,i}^c), \quad (26) \\ \mathcal{L}_{k-1,i}^c(\hat{d}_{k-1,i}^c) &= \underbrace{\ell_{\text{match},k-1,i}^c(\hat{d}_{k-1,i}^c)}_{\text{local cost matching}} \\ &\quad + \underbrace{\ell_{\text{reg},k-1,i}^c(\hat{d}_{k-1,i}^c)}_{\text{disturbance regularization}}, \\ \ell_{\text{match},k-1,i}^c(\hat{d}_{k-1,i}^c) &= \sum_{j \in \mathcal{N}_i} W_{ij}^c \left[e_{k,i,j}^c(\hat{d}_{k-1,i}^c) \right]^2, \\ \ell_{\text{reg},k-1,i}^c(\hat{d}_{k-1,i}^c) &= \left\| W_d^c \hat{d}_{k-1,i}^c \right\|_2^2. \end{aligned}$$

The matching term measures how well the candidate transports the previous local cost pattern to the current update. The weights $W_{ij}^c \geq 0$ set the importance of neighboring query states, with $\sum_{j \in \mathcal{N}_i} W_{ij}^c = 1$, while W_d^c regularizes the magnitude and coordinates of the candidate disturbance.

For a query-only or black-box cost interface, Eq. (26) is approximated by testing m_c candidate disturbances, sampled or placed on a finite local grid, and retaining the lowest-loss candidate at each query state. This search requires only cost queries and provides estimates at the N_c query states. To evaluate the correction at arbitrary rollout states, these estimates are interpolated with a confidence-weighted normalized kernel estimator (Nadaraya 1964; Kang and Joseph 2016):

$$\begin{aligned} Z_c(\mathbf{q}) &= \sum_{j=1}^{N_c} W_c^{(j)} K \left(\|\mathbf{q} - \mathbf{q}_k^{(j)}\| / \sigma_c \right), \\ \alpha_i(\mathbf{q}) &= \frac{W_c^{(i)} K \left(\|\mathbf{q} - \mathbf{q}_k^{(i)}\| / \sigma_c \right)}{Z_c(\mathbf{q})}, \end{aligned}$$

$$\hat{d}_{k-1}^c(\mathbf{q}) = \sum_{i=1}^{N_c} \alpha_i(\mathbf{q}) \hat{d}_{k-1,i}^c. \quad (27)$$

Here $W_c^{(i)}$ is the confidence assigned to query state i , K is a compact local kernel, and σ_c is its bandwidth. If $Z_c(\mathbf{q}) = 0$, the disturbance estimate is set to zero. Using a frozen local field and first-order backtracing, the cost query over the prediction horizon is

$$\hat{c}_{k+h|k}^c(\mathbf{q}) = c_k \left(\mathbf{q} - h \Delta t \hat{d}_{k-1}^c(\mathbf{q}) \right), \quad h = 1, \dots, H. \quad (28)$$

Thus $\hat{c}_{k+h|k}^c$ is the current query function transported by the latest apparent cost motion. Unexplained changes remain in the matching residual. No obstacle identity or object tracker is required.

4.5 Using MDOB in Control

Figure. 6 and Algorithm 1 summarize how MDOB enters the control update. At step k , $\hat{d}_{k-1}^c$ compensates the state rollouts, and their query states are evaluated with $\hat{c}_{k+h|k}^c$. Let

$$U_k = (\mathbf{u}_{k|k}, \dots, \mathbf{u}_{k+H-1|k}) \quad (29)$$

be a horizon control sequence. The controller solves

$$\begin{aligned} U_k^* &= \arg \min_{U_k} S_k(U_k), \quad (30) \\ S_k(U_k) &= \underbrace{\Phi_{\text{cost},k}(U_k)}_{\text{cost channel term}} + \underbrace{\Phi_{\text{other},k}(U_k)}_{\text{other constraint cost}} + \\ &\quad \underbrace{\Phi_{g,k}(U_k)}_{\text{goal cost}} + \underbrace{\Phi_{u,k}(U_k)}_{\text{u smooth}}, \\ \Phi_{\text{cost},k}(U_k) &= \sum_{h=1}^H \hat{c}_{k+h|k}^c(\mathbf{q}_{k+h|k}), \\ \Phi_{\text{other},k}(U_k) &= \sum_{h=0}^{H-1} \ell_k^{\text{con}}(\mathbf{x}_{k+h|k}, \mathbf{u}_{k+h|k}), \\ \Phi_{g,k}(U_k) &= F_c^f(\mathbf{x}_{k+H|k}, \mathbf{x}_g), \end{aligned}$$

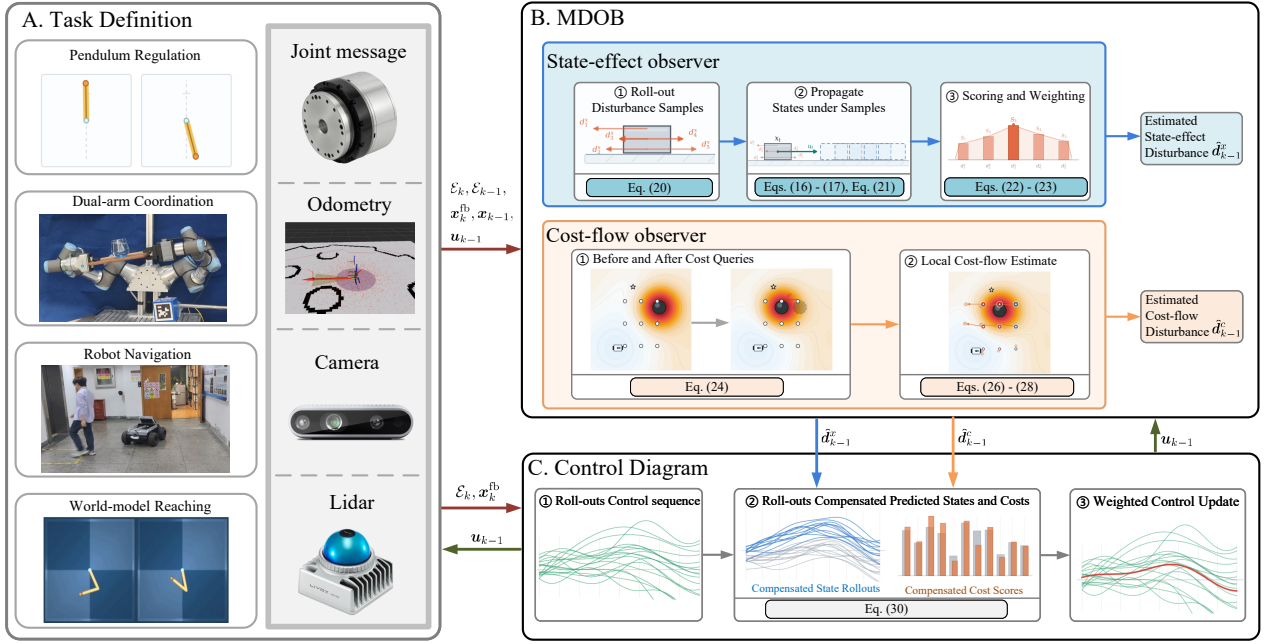


Figure 6. Computation flow of MDOB. Feedback updates the two disturbance channels, and their estimates modify subsequent state rollouts and cost scoring.

Algorithm 1: MDOB control update

Input: $\mathbf{x}_{k-1}$, $\mathbf{u}_{k-1}$, $\mathbf{x}_k^{\text{fb}}$, c_{k-1} , and c_k

Output: $\hat{\mathbf{d}}_{k-1}^x$, $\hat{\mathbf{d}}_{k-1}^c(\cdot)$, and $\mathbf{u}_k$

- 1 Sample state-effect candidates using (20);
 - 2 Estimate $\hat{\mathbf{d}}_{k-1}^x$ using (21)–(23);
 - 3 Evaluate paired cost queries using (25);
 - 4 Estimate local cost disturbances by solving (26) over finite candidates;
 - 5 Interpolate $\hat{\mathbf{d}}_{k-1}^c(\cdot)$ using (27);
 - 6 Transport the horizon cost query using (28);
 - 7 Solve (30) for $\mathbf{U}_k^*$;
 - 8 Apply $\mathbf{u}_k = \mathbf{u}_{k|k}^*$;
-

$$\Phi_{u,k}(U_k) = \sum_{h=0}^{H-1} \ell_k^{u,\text{sm}}(\Delta \mathbf{u}_{k+h|k}),$$

$$\text{s.t. } \mathbf{x}_{k|k} = \mathbf{x}_k^{\text{fb}},$$

$$\mathbf{x}_{k+h+1|k} = F_x^d(\mathbf{x}_{k+h|k}, \mathbf{u}_{k+h|k}, \hat{\mathbf{d}}_{k-1}^x),$$

$$\mathbf{q}_{k+h|k} = \Pi_q(\mathbf{x}_{k+h|k}).$$

The objective contains four terms. The cost channel term $\Phi_{\text{cost},k}$ evaluates costs exposed through the cost-query interface, such as collision cost, at each predicted task query state using the transported query function. The term $\Phi_{\text{other},k}$ collects constraints not handled through this interface, such as state or input limits, and is not modified by cost disturbance compensation. The goal term $\Phi_{g,k}$ measures the predicted goal error, while $\Phi_{u,k}$ penalizes changes between adjacent inputs to promote smooth control commands.

5 Simulation and Experiments

The experiments examine whether the two MDOB channels remain useful across different model classes and deployment conditions. Three simulation tasks provide controlled comparisons, while two real-robot tasks test closed-loop operation. The inverted pendulum and latent Reacher isolate state-effect disturbances, whereas navigation evaluates both channels. The hardware studies are representative demonstrations and are not intended as statistical measurements of the complete sim-to-real gap.

For fair comparison, all methods within each simulation task share the same nominal model, task cost, action limits, initial conditions, evaluation seeds, prediction horizon, and nominal rollout budget. Repeated runs use paired task or planner seeds, and method-specific settings remain fixed. MDOB is compared with the corresponding nominal controller and, when available, DOB, Tube-MPPI, and RMPPI. Reported planning time includes controller-specific observer and robustness computations. All simulation runs and real-robot controller computations are executed on a system equipped with an NVIDIA GeForce RTX 4080 Laptop GPU. Table 1 lists the core simulation settings; the remaining observer and controller parameters are provided in the open-source configuration files.

Closed-loop task performance is assessed using success rate together with task-specific tracking and safety metrics. Command smoothness is quantified by control total variation

Table 1. Core MDOB and controller parameters used in simulation. The navigation grid contains 332 active query states. A dash means that the channel is not used.

Task	MDOB channel	MDOB state samples m_x	MDOB query states N_c	Controller rollout samples	Prediction horizon H	Environment step
Inverted pendulum	State effect	256	–	1024	15	0.05 s
Eight-obstacle navigation	State and cost	256	332	500	30	0.05 s
Latent Reacher	State effect	256	–	300	5	0.02 s

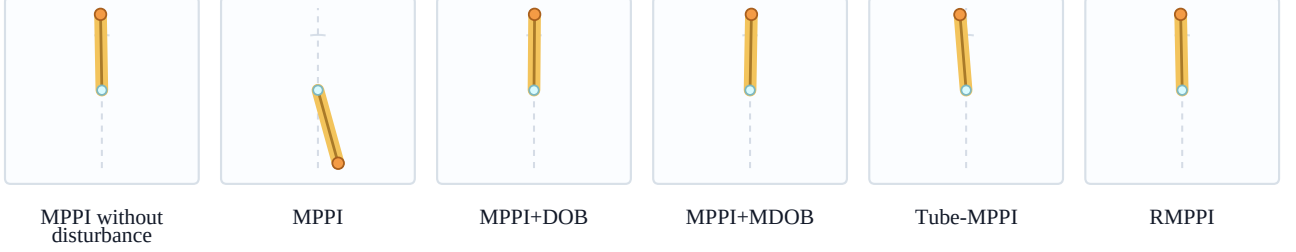


Figure 7. Representative inverted-pendulum snapshots under a strong sinusoidal torque bias.

Table 2. Inverted-pendulum simulation results.

Method	Success	Tail $ \theta $ (rad)	Settling time (s)	Disturbance RMSE	Control TV	P95 planning time (ms)
MPPI (no disturbance)	20/20	0.008 ± 0.004	0.897 ± 0.203	–	0.125 ± 0.009	8.274 ± 0.541
MPPI	0/20	1.108 ± 0.006	–	–	0.172 ± 0.010	8.024 ± 0.462
MPPI+DOB	20/20	0.027 ± 0.004	2.413 ± 0.142	0.018 ± 0.001	0.127 ± 0.007	8.018 ± 0.405
MPPI+MDOB	20/20	0.026 ± 0.004	2.402 ± 0.157	0.014 ± 0.001	0.127 ± 0.007	8.232 ± 0.426
Tube-MPPI	19/20	0.081 ± 0.008	2.668 ± 0.662	–	0.162 ± 0.007	16.914 ± 1.192
RMPPI	20/20	0.021 ± 0.006	2.423 ± 0.227	–	0.178 ± 0.010	19.628 ± 1.143

(TV),

$$\text{TV}(\mathbf{u}) = \frac{1}{T-1} \sum_{k=0}^{T-2} \|\mathbf{u}_{k+1} - \mathbf{u}_k\|_2, \quad (31)$$

where a lower value indicates smaller changes between consecutive commands. Computational latency is summarized by the 95th percentile (P95) of per-step planning time, which captures the upper tail of controller runtime. Continuous values are reported as mean $\pm$ standard deviation.

5.1 Simulation Results

1) Inverted-pendulum state-effect task: The state-effect channel is first evaluated on an inverted pendulum with a sinusoidal torque disturbance. Figure 7 shows representative snapshots under the common disturbance, and Table 2 summarizes 20 paired sampling seeds.

Except for the disturbance-free nominal reference, all methods are evaluated under the same sinusoidal torque disturbance, with success requiring $|\theta| < 0.12$ rad for at least the final 20% of the rollout. The disturbance acts through the modeled torque channel and its value is known for evaluation, so this task tests matched state-effect estimation rather than arbitrary unmatched dynamics. The uncompensated MPPI controller fails in all 20 trials, whereas MPPI+DOB, MPPI+MDOB, and RMPPI succeed

in all trials and Tube-MPPI succeeds in 19. MPPI+DOB and MPPI+MDOB have similar tail error and control variation, but MPPI+MDOB reduces the disturbance estimation RMSE from 0.018 to 0.014. RMPPI achieves a smaller tail error at more than twice the P95 planning time of MPPI+MDOB.

2) Eight-obstacle navigation with state-effect and cost disturbances: The two channels are evaluated jointly in an eight-obstacle navigation task. A constant input bias changes the robot motion, while moving obstacles change the queried costs. The obstacles approach from several directions and may overlap. All controllers use the same obstacle margin. Figure 8 shows six instants, and Table 3 reports 20 paired sampling seeds.

Each run has a 30 s limit, and success means reaching the 0.35 m goal region before collision. The path ratio is the traveled path length divided by the straight-line distance from the start to the goal, so a lower value indicates a more direct route. Goal time and path ratio use successful trials only; negative clearance denotes overlap. MPPI, MPPI+DOB, and Tube-MPPI collide in all trials, whereas MPPI+MDOB and RMPPI succeed in all trials. MPPI+MDOB has a shorter mean path ratio than RMPPI with similar clearance, while its local query update gives the largest planning time.

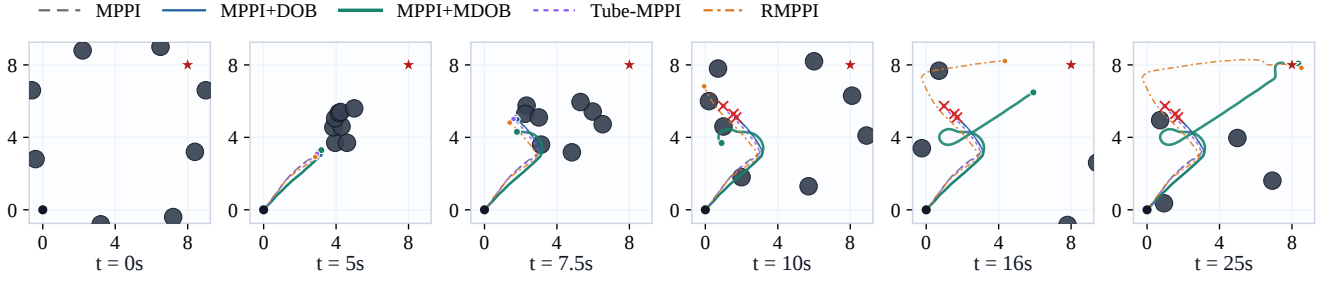


Figure 8. Eight-obstacle navigation under state-effect and cost disturbances. Red crosses indicate collisions.

Table 3. Eight-obstacle navigation simulation results.

Method	Success	Collision	Min. clearance (m)	Goal time (s)	Path ratio	Control TV	P95 planning time (ms)
MPPI	0/20	20/20	-0.006 ± 0.004	—	—	0.697 ± 0.053	7.368 ± 0.708
MPPI+DOB	0/20	20/20	-0.009 ± 0.004	—	—	0.689 ± 0.051	8.736 ± 1.410
MPPI+MDOB	20/20	0/20	0.129 ± 0.034	21.323 ± 2.846	1.484 ± 0.171	0.554 ± 0.034	20.770 ± 1.661
Tube-MPPI	0/20	20/20	-0.002 ± 0.002	—	—	0.674 ± 0.046	7.771 ± 0.889
RMPPI	20/20	0/20	0.125 ± 0.010	21.225 ± 0.026	1.573 ± 0.003	0.518 ± 0.025	18.764 ± 1.146

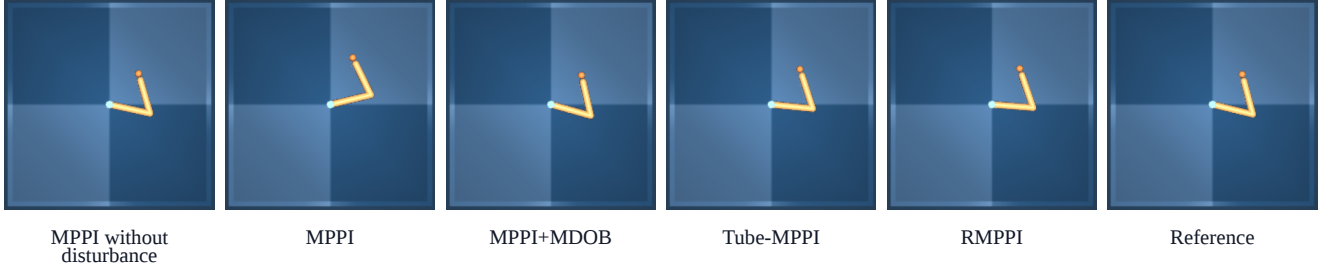


Figure 9. Final Reacher observations from representative evaluations. The left panel shows the no-disturbance nominal planner, the middle panels show controllers under persistent action bias, and the rightmost panel shows the reference image.

Table 4. Latent Reacher simulation results over 20 paired task seeds under persistent action bias. Continuous values are reported as mean $\pm$ standard deviation.

Method	Success	Tail mean error (rad)	Final error (rad)	Disturbance RMSE	Control TV	P95 planning time (ms)
MPPI (no disturbance)	20/20	0.017 ± 0.004	0.016 ± 0.007	—	0.104 ± 0.011	438.956 ± 41.523
MPPI	0/20	0.427 ± 0.038	0.431 ± 0.038	—	0.125 ± 0.016	456.264 ± 69.320
MPPI+MDOB	19/20	0.030 ± 0.010	0.031 ± 0.012	0.056 ± 0.011	0.126 ± 0.020	502.569 ± 5.313
Tube-MPPI	0/20	0.185 ± 0.062	0.220 ± 0.105	—	0.163 ± 0.051	492.350 ± 38.858
RMPPI	0/20	0.176 ± 0.044	0.208 ± 0.109	—	0.152 ± 0.043	492.228 ± 41.545

3) *Latent world-model Reacher task:* The second state-effect study uses the pretrained Reacher checkpoint released with LeWorldModel (LeWM) (Maes et al. 2026). The controller encodes current and goal images, samples actions with MPPI, and scores latent rollouts. A constant action bias $[0.30, -0.20]$ is applied during execution but hidden from the planner. MDOB estimates an equivalent action bias from the latent endpoint mismatch. The controller uses images and latent states; joint angles are used only for evaluation. Because the model does not expose the local linear dynamics required by LQR, Tube-MPPI and RMPPI use MPPI as their auxiliary controller.

The task batch is conditioned on nominal solvability. We select 20 tasks for which disturbance-free MPPI has a final-20-step mean error no greater than 0.025 rad, then evaluate every disturbed method on the same tasks and planner seeds. Success requires this tail error to be below 0.05 rad. As shown in Figure 9 and Table 4, MPPI+MDOB succeeds in 19 trials and lowers the mean tail error to 0.030 rad, versus 0.427, 0.185, and 0.176 rad for MPPI, Tube-MPPI, and RMPPI. These results demonstrate the feasibility of integrating MDOB with a latent world model for online disturbance compensation.

Table 5. Completion times in the representative real-car runs.

Method	Completion time (s)
MPPI	12.13
MPPI+DOB	14.04
MPPI+MDOB	9.29
Tube-MPPI	12.07
RMPPI	13.10

5.2 Experimental Results

Two closed-loop hardware demonstrations test whether MDOB remains executable with real sensing, actuation, and changing environments. They report representative task outcomes, not repeated-run statistics, recovery of individual physical disturbances, or statistical superiority over the comparison methods.

1) *Real-car obstacle-avoidance navigation task*: Vehicle motion is compared with nominal rollouts, while a moving obstacle changes the perceived cost field. No ground-truth disturbance label is available, so the trial is assessed by route choice, collision avoidance, and completion time. In the representative runs in Figure 10, MPPI+MDOB follows the shortest route and finishes in 9.29 s, as listed in Table 5.

2) *Dual-arm obstacle-avoidance task*: The minimum collision distance is set to 0 cm, so no additional safety margin is imposed. The obstacle carries an AprilTag (Olson 2011) to update its current pose in the collision model, but no independent measurement of its Cartesian velocity or future trajectory is provided to the controller. MDOB instead estimates the obstacle motion from temporal changes in the collision-cost queries. In the representative trials in Figure 11, MPPI+MDOB begins avoidance earlier and avoids the obstacle, while the shown comparison runs collide.

6 Conclusion and Discussion

This paper introduced modern disturbance observers (MDOB) for models accessed through forward rollouts and cost queries. The state channel estimates a control-equivalent correction within a task-defined rollout interface, while the cost channel estimates local cost transport from repeated queries. The simulations show that these interface-level estimates can improve subsequent rollouts and cost scoring in the disturbances and tasks studied. The hardware demonstrations establish that the same update can run with real feedback. These results support online correction of observable model mismatch, but do not establish complete closure of the sim-to-real gap or universal superiority over robust controllers.

The method is limited by the information exposed through the chosen interfaces. State effects that do not influence the measured feedback cannot be identified. Cost transport depends on query spacing, update rate, and local cost structure, and becomes ambiguous in flat regions, under abrupt changes, or when several unresolved motions overlap. The current formulation assumes short-term persistence, tests only a matched action bias in the world-model task, and provides no general closed-loop stability guarantee. The hardware results are representative demonstrations rather than repeated statistical evidence. Future work will address multimodal transport, contact-driven object effects, uncertainty bounds, stability, and repeated hardware evaluation.

Data and Code Availability

Source code, configurations, experiment scripts, manuscript source, and figure-generation scripts are available at https://github.com/zhutianqi2006/2026_modern_dob.

Funding

Funding information will be completed before submission.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

References

- Abraham I, Handa A, Ratliff N, Lowrey K, Murphey TD and Fox D (2020) Model-based generalization under parameter uncertainty using path integral control. *IEEE Robotics and Automation Letters* 5(2): 2864–2871. <https://doi.org/10.1109/LRA.2020.2972836>.
- Ai B, Tian S, Shi H, Wang Y, Pfaff T, Tan C, Christensen HI, Su H, Wu J and Li Y (2025) A review of learning-based dynamics models for robotic manipulation. *Science Robotics* 10(106): adt1497. <https://doi.org/10.1126/scirobotics.adt1497>.
- Bardes A, Garrido Q, Ponce J, Chen X, Rabbat M, LeCun Y, Assran M and Ballas N (2024) Revisiting feature prediction for learning visual representations from video. <https://doi.org/10.48550/arXiv.2404.08471>.
- Carius J, Ranftl R, Farshidian F and Hutter M (2022) Constrained stochastic optimal control with learned importance sampling: A path integral approach. *The International Journal of Robotics Research* 41(2): 189–209. <https://doi.org/10.1177/02783649211047890>.



Figure 10. Representative real-car navigation snapshots for MPPI, MPPI+DOB, MPPI+MDOB, Tube-MPPI, and RMPPI.

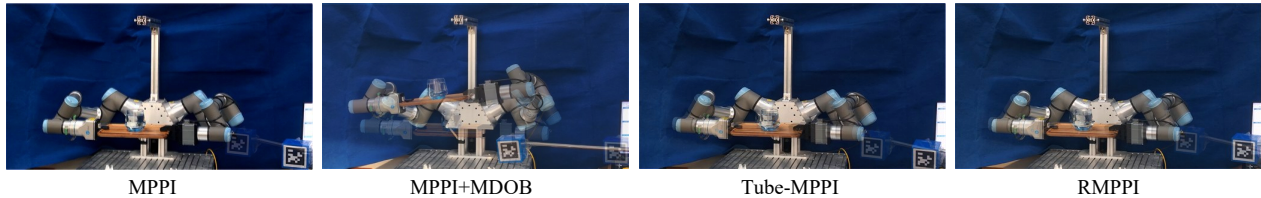


Figure 11. Representative dual-arm obstacle-avoidance snapshots for MPPI, MPPI+MDOB, Tube-MPPI, and RMPPI.

- Chebatar Y, Handa A, Makoviychuk V, Macklin M, Issac J, Ratliff N and Fox D (2019) Closing the sim-to-real loop: Adapting simulation randomization with real world experience. In: *2019 International Conference on Robotics and Automation (ICRA)*. pp. 8973–8979. <https://doi.org/10.1109/ICRA.2019.8793789>.
- Chen WH, Ballance DJ, Gawthrop PJ and O'Reilly J (2000a) A nonlinear disturbance observer for robotic manipulators. *IEEE Transactions on Industrial Electronics* 47(4): 932–938. <https://doi.org/10.1109/41.857974>.
- Chen WH, Yang J, Guo L and Li S (2016) Disturbance-observer-based control and related methods—an overview. *IEEE Transactions on Industrial Electronics* 63(2): 1083–1095. <https://doi.org/10.1109/TIE.2015.2478397>.
- Chen XK, Komada S and Fukuda T (2000b) Design of a nonlinear disturbance observer. *IEEE Transactions on Industrial Electronics* 47(2): 429–437. <https://doi.org/10.1109/41.836359>.
- Gandhi MS, Vlahov B, Gibson J, Williams G and Theodorou EA (2021) Robust model predictive path integral control: Analysis and performance guarantees. *IEEE Robotics and Automation Letters* 6(2): 1423–1430. <https://doi.org/10.1109/LRA.2021.3057563>.
- Gao Z (2003) Scaling and bandwidth-parameterization based controller tuning. In: *Proceedings of the 2003 American Control Conference*, volume 6. pp. 4989–4996. <https://doi.org/10.1109/ACC.2003.1242516>.
- Garcia CE, Prett DM and Morari M (1989) Model predictive control: Theory and practice—a survey. *Automatica* 25(3): 335–348. [https://doi.org/10.1016/0005-1098\(89\)90002-2](https://doi.org/10.1016/0005-1098(89)90002-2).
- Ha D and Schmidhuber J (2018) Recurrent world models facilitate policy evolution. In: *Advances in Neural Information Processing Systems*.
- Hafner D, Lillicrap T, Ba J and Norouzi M (2020) Dream to control: Learning behaviors by latent imagination. In: *International Conference on Learning Representations (ICLR)*. URL <https://openreview.net/forum?id=S110TC4tDS>.
- Hafner D, Lillicrap T, Fischer I, Villegas R, Ha D, Lee H and Davidson J (2019) Learning latent dynamics for planning from pixels. In: *Proceedings of the 36th International Conference on Machine Learning, Proceedings of Machine Learning Research*, volume 97. pp. 2555–2565.
- Hafner D, Pasukonis J, Ba J and Lillicrap T (2025) Mastering diverse control tasks through world models. *Nature* 640(8059): 647–653. <https://doi.org/10.1038/s41586-025-08744-2>.
- Han J (2009) From PID to active disturbance rejection control. *IEEE Transactions on Industrial Electronics* 56(3): 900–906. <https://doi.org/10.1109/TIE.2008.2011621>.
- Hansen N, Su H and Wang X (2024) TD-MPC2: Scalable, robust world models for continuous control. In: *International Conference on Learning Representations (ICLR)*.
- Hornung A, Wurm KM, Bennewitz M, Stachniss C and Burgard W (2013) OctoMap: An efficient probabilistic 3D mapping framework based on octrees. *Autonomous Robots* 34(3): 189–206. <https://doi.org/10.1007/s10514-012-9321-0>.
- Huang W, Chao YW, Mousavian A, Liu MY, Fox D, Mo K and Li FF (2026) PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. <https://doi.org/10.48550/arXiv.2601.03782>.
- Jaquier N, Welle MC, Gams A, Yao K, Fichera B, Billard A, Ude A, Asfour T and Kragić D (2025) Transfer learning in robotics: An upcoming breakthrough? a review of promises and challenges.

- The International Journal of Robotics Research* 44(3): 465–485. <https://doi.org/10.1177/02783649241273565>.
- Kadian A, Truong J, Gokaslan A, Clegg A, Wijmans E, Lee S, Savva M, Chernova S and Batra D (2020) Sim2Real predictivity: Does evaluation in simulation predict real-world performance? *IEEE Robotics and Automation Letters* 5(4): 6670–6677. <https://doi.org/10.1109/LRA.2020.3013848>.
- Kang L and Joseph VR (2016) Kernel approximation: From regression to interpolation. *SIAM/ASA Journal on Uncertainty Quantification* 4(1): 112–129. <https://doi.org/10.1137/130948112>.
- Kobilarov M (2012) Cross-entropy motion planning. *The International Journal of Robotics Research* 31(7): 855–871. <https://doi.org/10.1177/0278364912444543>.
- Koptev M, Figueroa N and Billard A (2024) Reactive collision-free motion generation in joint space via dynamical systems and sampling-based MPC. *The International Journal of Robotics Research* 43(13): 2049–2069. <https://doi.org/10.1177/02783649241246557>.
- LeCun Y (2022) A path towards autonomous machine intelligence. OpenReview position paper.
- Maes L, Le Lidec Q, Scieur D, LeCun Y and Balestrieri R (2026) LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint* <https://doi.org/10.48550/arXiv.2603.19312>.
- Mayne DQ, Rawlings JB, Rao CV and Scokaert POM (2000) Constrained model predictive control: Stability and optimality. *Automatica* 36(6): 789–814. [https://doi.org/10.1016/S0005-1098\(99\)00214-9](https://doi.org/10.1016/S0005-1098(99)00214-9).
- Mohammadi A, Tavakoli M, Marquez HJ and Hashemzadeh F (2013) Nonlinear disturbance observer design for robotic manipulators. *Control Engineering Practice* 21(3): 253–267. <https://doi.org/10.1016/j.conengprac.2012.10.008>.
- Nadaraya EA (1964) On estimating regression. *Theory of Probability and Its Applications* 9(1): 141–142. <https://doi.org/10.1137/1109020>.
- Ohnishi K, Shibata M and Murakami T (1996) Motion control for advanced mechatronics. *IEEE/ASME Transactions on Mechatronics* 1(1): 56–67. <https://doi.org/10.1109/3516.491410>.
- Oleynikova H, Taylor Z, Fehr M, Siegwart R and Nieto J (2017) Voxblox: Incremental 3D euclidean signed distance fields for on-board MAV planning. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 1366–1373. <https://doi.org/10.1109/IROS.2017.8202315>.
- Olson E (2011) AprilTag: A robust and flexible visual fiducial system. In: *2011 IEEE International Conference on Robotics and Automation*. pp. 3400–3407. <https://doi.org/10.1109/ICRA.2011.5979561>.
- OpenAI, Andrychowicz M, Baker B, Chociej M, Józefowicz R, McGrew B, Pachocki J, Petron A, Plappert M, Powell G, Ray A, Schneider J, Sidor S, Tobin J, Welinder P, Weng L and Zaremba W (2020) Learning dexterous in-hand manipulation. *The International Journal of Robotics Research* 39(1): 3–20. <https://doi.org/10.1177/0278364919887447>.
- Ostafew CJ, Schoellig AP and Barfoot TD (2016) Robust constrained learning-based NMPC enabling reliable mobile robot path tracking. *The International Journal of Robotics Research* 35(13): 1547–1563. <https://doi.org/10.1177/0278364916645661>.
- Park G and Back J (2024) Survey of disturbance observer-based controller and related challenging problems. *Journal of Institute of Control, Robotics and Systems* 30(4): 363–372. <https://doi.org/10.5302/J.ICROS.2024.24.0045>.
- Pinneri C, Sawant S, Blaes S, Achterhold J, Stueckler J, Rolinek M and Martius G (2021) Sample-efficient cross-entropy method for real-time planning. In: *Proceedings of the 2020 Conference on Robot Learning, Proceedings of Machine Learning Research*, volume 155. pp. 1049–1065.
- Rubinstein RY (1997) Optimization of computer simulation models with rare events. *European Journal of Operational Research* 99(1): 89–112. [https://doi.org/10.1016/S0377-2217\(96\)00385-2](https://doi.org/10.1016/S0377-2217(96)00385-2).
- Sariyildiz E, Oboe R and Ohnishi K (2020) Disturbance observer-based robust control and its applications: 35th anniversary overview. *IEEE Transactions on Industrial Electronics* 67(3): 2042–2053. <https://doi.org/10.1109/TIE.2019.2903752>.
- Shim H and Jo NH (2009) An almost necessary and sufficient condition for robust stability of closed-loop systems with disturbance observer. *Automatica* 45(1): 296–299. <https://doi.org/10.1016/j.automatica.2008.10.009>.
- Tan J, Zhang T, Coumans E, Iscen A, Bai Y, Hafner D, Bohez S and Vanhoucke V (2018) Sim-to-real: Learning agile locomotion for quadruped robots. In: *Proceedings of Robotics: Science and Systems*. Pittsburgh, Pennsylvania. <https://doi.org/10.15607/RSS.2018.XIV.010>.
- Theodorou E, Buchli J and Schaal S (2010) A generalized path integral control approach to reinforcement learning. *Journal of Machine Learning Research* 11: 3137–3181.
- Tobin J, Fong R, Ray A, Schneider J, Zaremba W and Abbeel P (2017) Domain randomization for transferring deep neural networks from simulation to the real world. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 23–30. <https://doi.org/10.1109/IROS.2017.8202133>.

- Wang G, Ren K, Morgan AS and Hang K (2026) ManiDreams: An open-source library for robust object manipulation via uncertainty-aware task-specific intuitive physics. <https://doi.org/10.48550/arXiv.2603.18336>.
- Williams G, Aldrich A and Theodorou EA (2015) Model predictive path integral control using covariance variable importance sampling. <https://doi.org/10.48550/arXiv.1509.01149>.
- Williams G, Drews P, Goldfain B, Rehg JM and Theodorou EA (2018a) Information-theoretic model predictive control: Theory and applications to autonomous driving. *IEEE Transactions on Robotics* 34(6): 1603–1622. <https://doi.org/10.1109/TRO.2018.2865891>.
- Williams G, Goldfain B, Drews P, Saigol K, Rehg JM and Theodorou EA (2018b) Robust sampling based model predictive control with sparse objective information. In: *Proceedings of Robotics: Science and Systems*. Pittsburgh, Pennsylvania. <https://doi.org/10.15607/RSS.2018.XIV.042>.
- Yang J, Zheng WX, Li S, Wu B and Cheng M (2015) Design of a prediction-accuracy-enhanced continuous-time MPC for disturbed systems via a disturbance observer. *IEEE Transactions on Industrial Electronics* 62(9): 5807–5816. <https://doi.org/10.1109/TIE.2015.2450736>.
- Yin J, Dawson C, Fan C and Tsiotras P (2023) Shield model predictive path integral: A computationally efficient robust MPC method using control barrier functions. *IEEE Robotics and Automation Letters* 8(11): 7106–7113. <https://doi.org/10.1109/LRA.2023.3315211>.
- Yoneda T, Yang G, Walter MR and Stadie BC (2022) Invariance through latent alignment. In: *Proceedings of Robotics: Science and Systems*. New York City, NY, USA. <https://doi.org/10.15607/RSS.2022.XVIII.064>.
- Zucker M, Ratliff N, Dragan AD, Pivtoraiko M, Klingensmith M, Dellin CM, Bagnell JA and Srinivasa SS (2013) CHOMP: Covariant hamiltonian optimization for motion planning. *The International Journal of Robotics Research* 32(9–10): 1164–1193. <https://doi.org/10.1177/0278364913488805>.